

CDGC Data Element Lineage Pipeline — Table Overview

Summary of each table in the data element lineage flow: what it does, what it stores, how to query it, and the known challenges in reading it for proper lineage.

1. CDGC_DATA_ELEMENT_LINEAGE (Raw)

What it does

Raw landing table. Loaded daily by the IICS job (tf_CDGC_CDQ_Asset_Lineage), triggered via ESP, pulling from the CDGC bulk API. Reloaded per SEED_OBJECT_ASSET_ID — all rows for a given seed ID are deleted and reinserted on each load.

What it stores

- SEED_OBJECT_ASSET_ID, SEED_ASSET_PATH, SEED_ASSET_TYPE — the anchor asset for this lineage
- ASSET_ID, ASSET_PATH, ASSET_TYPE — the related asset at this hop
- LINEAGE_LEVEL — a comma-separated string of level numbers (not a single value)
- IMMEDIATE_DOWNSTREAM, IMMEDIATE_UPSTREAM — comma-separated strings of related asset paths
- CREATED_DTTM — load timestamp, used as the change-detection watermark

How to query it

Generally not queried directly for lineage — the multi-value columns need LATERAL FLATTEN to split LINEAGE_LEVEL and IMMEDIATE_DOWNSTREAM/UPSTREAM into individual rows before the data is usable.

Challenges for proper lineage

1. Multi-value strings, not normalized rows — a single row can represent multiple hops at once, packed into one comma-separated field. Must flatten before treating it as one-row-per-relationship.
2. No direct positional link between LINEAGE_LEVEL and IMMEDIATE_DOWNSTREAM/UPSTREAM — flattening both independently produces a cross join between every level value and every downstream/upstream path in that row.
3. No built-in stopping logic — the raw data includes every hop the source system found, with no way to flag a node as a dead end (e.g. AUDIT_BALANCE at dataset level); that has to be layered on downstream.
4. Change detection isn't explicit — nothing in the table states which seed IDs changed on a given load; this is inferred from CREATED_DTTM against a separately maintained watermark.

2. CDGC_DATA_ELEMENT_LINEAGE_FLAT (was STG)

What it does

Unrolls the raw comma-separated fields into one row per hop, split OUTBOUND/INBOUND, via two LATERAL FLATTENS (on LINEAGE_LEVEL and on IMMEDIATE_DOWNSTREAM/UPSTREAM) plus a self-join back to the raw table to resolve CHILD_ASSET_ID.

What it stores

- All raw columns, plus LINEAGE_LEVEL_RAW (the original comma-separated string) and a parsed integer LINEAGE_LEVEL
- LINEAGE_DIRECTION — 'OUTBOUND' or 'INBOUND', one row set built per direction via UNION ALL
- CHILD_ASSET_PATH — the flattened downstream/upstream path for this hop
- CHILD_ASSET_ID — resolved via self-join back to the raw table
- IS_VIEW_ROW — flags whether the asset at this hop is a database view

How to query it

Queryable directly as one-row-per-hop lineage data, filtered by SEED_OBJECT_ASSET_ID, LINEAGE_DIRECTION, and LINEAGE_LEVEL as needed.

Challenges for proper lineage

1. Inherits the raw table's cross-join behavior — the two LATERAL FLATTENS are independent, so every LINEAGE_LEVEL value is paired with every CHILD_ASSET_PATH value in that row. Confirmed intentional/expected, but not obvious from the table alone.
2. Still contains the full hop list with no stopping logic — a node like AUDIT_BALANCE still generates further downstream rows in this table; nothing here prevents continued traversal past it.
3. A full rebuild (CREATE OR REPLACE TABLE, or currently a view) reprocesses all seed IDs even when only a small subset changed in the raw table that day.

3. CDGC_DATA_ELEMENT_LINEAGE_DERIVED (was CURATED)

What it does

Classifies each node as ROOT / INTERMEDIATE / LEAF (checking the raw table's IMMEDIATE_DOWNSTREAM/UPSTREAM directly), builds ULTIMATE_SOURCE/ULTIMATE_TARGET via LISTAGG, and generates RECURSIVE_FROM_NODE_ID/RECURSIVE_TO_NODE_ID (MD5 hashes) — this is the edge list a downstream graph traversal consumes.

What it stores

- SOURCE_ASSET_ID / TARGET_ASSET_ID — the two ends of each lineage relationship
- SOURCE_NODE_TYPE / TARGET_NODE_TYPE — ROOT, INTERMEDIATE, or LEAF classification
- ULTIMATE_SOURCE / ULTIMATE_TARGET — aggregated list of true leaf-node paths per seed
- RECURSIVE_FROM_NODE_ID / RECURSIVE_TO_NODE_ID — MD5-hashed node identifiers used as graph edges by the recursive path builder
- MAPPING_NAME / MAPPING_PATH — details of any ETL bridging asset between two real nodes

How to query it

Queried as a graph edge list — typically consumed by the recursive CTE rather than queried standalone, though it can be filtered by SEED_OBJECT_ASSET_ID and NODE_TYPE directly.

Challenges for proper lineage

1. LEAF classification is based on whether the base table shows further downstream/upstream data, not on business-defined stopping points — a node can have real further lineage in the source system but still need to be treated as a terminus for practical reporting purposes (e.g. AUDIT_BALANCE at dataset level).
2. This is dataset-level specific — data element (column-level) lineage does not surface AUDIT_BALANCE at all, since it's a dataset-level table reference.
3. Same full-rebuild cost concern as FLAT — every self-join, DENSE_RANK(), and LISTAGG() recomputes across all seed IDs on each rebuild unless scoped to changed IDs only.

4. CDGC_DATA_ELEMENT_LINEAGE_RECURSIVE_PATH (Recursive CTE output)

What it does

Walks the edges from the DERIVED table using WITH RECURSIVE, building full LINEAGE_PATH strings hop by hop in both OUTBOUND and INBOUND directions, capped at PATH_LEVEL < 50. Not yet moved to production.

What it stores

- PATH_LEVEL — how many hops deep this row represents
- LINEAGE_PATH — the full chain of asset paths from seed to this point, built incrementally via string concatenation
- VISITED_NODE_IDS (proposed addition) — array of every node ID seen so far in this specific path, used for cycle detection
- All columns carried through from the DERIVED table at each step (SOURCE/TARGET IDs, node types, lineage direction)

How to query it

Queried as fully-built lineage chains per seed asset — this is the table meant for direct consumption by reporting or UI, since the recursion work is already done.

Challenges for proper lineage

1. Cycle risk — circular lineage can occur in this data (feedback loops, shared audit/reconciliation tables). Without cycle detection, a loop would recurse until hitting the PATH_LEVEL < 50 cap, producing incorrect looping/duplicated paths rather than stopping cleanly.
2. The PATH_LEVEL < 50 cap alone is a depth safety net, not a correctness guarantee — it prevents infinite recursion but does not prevent incorrect intermediate output from a cycle.

3. Recommended fix (not yet applied): track VISITED_NODE_IDS per path and block the recursive step from revisiting a node already in that array, stopping the walk the moment a cycle would occur instead of relying on the depth cap.
4. Re-running this recursive query against a live view (rather than a materialized DERIVED table) would recompute the entire FLAT/DERIVED transformation chain on every one of the up to 50 recursion steps — a significant performance risk once this moves forward.